

CPRE 4880 - Section A1

MP - 3

Cameron Gilbertson, Matthew Smith, Jessica Reiff, Collin Van Weelden

12 March 2026

Dr. Phillip Jones

Table of Contents

Getting Started.....	3
In your write-up, briefly explain how this Make process was configured to appropriately use a cross-compiler targeting the ARM architecture.....	3
Booting Linux.....	3
In your write-up, provide an annotation explaining some of the boot messages that print out as Linux is booting on your ZedBoard.....	3
Phase 1: U-Boot Bootloader.....	3
Phase 2: Linux Kernel Initialization.....	4
Phase 3: UserSpace Initialization.....	5
Include these kernel messages in your writeup and attempt to provide some meaning to the output.....	5
Controlling Simple Peripherals.....	7
USB Driver Development.....	8
In your write-up, summarize the changes that you've made.....	8
In your writeup, describe how launcher_fire.c generally works.....	8
Sentry System.....	10
In your write-up, describe the general algorithm you implemented.....	10

Getting Started

In your write-up, briefly explain how this Make process was configured to appropriately use a cross-compiler targeting the ARM architecture.

The PetaLinux toolchain automatically configures the cross-compiler through its environment setup script. Running `source /remote/petalinux/settings.sh` at the start of each session configures the shell environment with the appropriate `ARCH` and `CROSS_COMPILE` variables, pointing Make to the ARM cross-compiler bundled with PetaLinux rather than the host machine's native x86 compiler. This means all compiled output, the kernel, device drivers, and user applications, target the ARM Cortex-A9 cores of the Zynq 7z020 on the ZedBoard.

The hardware platform was then imported using `petalinux-config --get-hw-description`, which ingests the .xsa file exported from Vivado and configures the build system to match the specific peripherals and memory map of our hardware. From there, `petalinux-build -c kernel` compiled the kernel, and `petalinux-build -c launcher-driver` compiled our kernel module, both using the ARM cross-compiler transparently. User applications were similarly compiled with `petalinux-build -c myapp`, producing ARM ELF executables found under the `cortexa9t2hf-neon-xilinx-linux-gnueabi` target directory, confirming the correct architecture was targeted throughout.

Booting Linux

In your write-up, provide an annotation explaining some of the boot messages that print out as Linux is booting on your ZedBoard.

We found that the specific lines correspond to the different phases of Linux booting, so we organized them below.

Phase 1: U-Boot Bootloader

Line	Annnotation
U-Boot 2020.01 (Mar 20 2026 - 17:05:22 +0000)	Universal Bootloader version 2020.01 begins execution. The timestamp is referencing the compilation of the binary file.
CPU: Zynq 7z020	The processor is identified as being the Zync 7z020

Model: Zynq Zed Development Board	U-Boot confirms this is the board it is initializing on.
Scanning mmc 0:1...	U-Boot is scanning mmc (multi-media card) device 0, partition 1 (the SD card), looking for a boot script.
Found U-Boot script /boot.scr	U-Boot found a boot script named boot.scr on the SD card.
## Executing script at 03000000	U-Boot loaded the boot.scr into RAM at this address and is executing the contents.
## Loading kernel from FIT Image at 10000000 ...	U-Boot loads the kernel from a FIT (Flattened Image Tree) located in an image at this RAM address.
Using 'conf@system-top.dtb' configuration	U-Boot has selected the config entry 'system-top.dtb'.
Trying 'ramdisk@1' ramdisk subimage	The U-Boot is trying to load the ramdisk subimage from the FIT.
Starting kernel ...	U-Boot is finished and is no longer active; it passes control to the Linux Kernel.

Phase 2: Linux Kernel Initialization

Line	Annotation
Booting Linux on physical CPU 0x0	Linux begins executing on Core 0
CPU: ARMv7 Processor [413fc090] revision 0 (ARMv7), cr=18c5387d	The kernel is reading the CPU identity register. Cr (control register) shows which CPU features are currently enabled.
Kernel command line: console=ttyPS0,115200 earlycon root=/dev/ram0 rw	This is the kernel command-line parameters. The output is to be sent to ttyPS0 (UART 1) at a Baud Rate of 115200, and /dev/ram0 is where to find the root filesystem, where it is read/write
Memory: 484532K/524288K available (6144K kernel code, 217K rwddata, 1840K rodata, 1024K init, 131K bss, 23372K reserved, 16384K cma-reserved, 0K highmem)	Of the Zedboards' 524288KiB of RAM, 484532KiB is available for processes.
sched_clock: 64 bits at 333MHz, resolution 3ns, wraps every 4398046511103ns	The kernel clock is sourced from a timer running at 333MHz. It has 3ns of resolution

XGpio: gpio@41200000: registered, base is 1016	There is a GPIO controller with address space starting at 0x41200000. 1016 is the starting index in the Linux numbering space.
spi-nor spi0.0: found s25fl256s1, expected n25q128a11	The device tree does not match the board's actual flash chip.
spi-nor spi0.0: s25fl256s1 (32768 Kbytes)	The spi-nor flash drive has been successfully identified.
Creating 8 MTD partitions on "spi0.0":	The kernel divides the SPI into MTD (Memory Technology Device) partitions. This allows for individual read/writes access to each flash region.

Phase 3: UserSpace Initialization

Line	Annotation
ULPI transceiver vendor/product ID 0x0451/0x1507	The physical chip vendor and product ID
ci_hdrc ci_hdrc.0: USB 2.0 started, EHCI 1.00	The controller driver is initialized and ready
usb usb1: New USB device found, idVendor=1d6b, idProduct=0002, bcdDevice= 5.04	A new USB has been found on the USB bus.

Include these kernel messages in your writeup and attempt to provide some meaning to the output.

```
usb 1-1: new low-speed USB device number 2 using ci_hdrc
usb 1-1: new low-speed USB device number 3 using ci_hdrc
usb 1-1: New USB device found, idVendor=2123, idProduct=1010, bcdDevice= 0.01
usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
usb 1-1: Product: USB Missile Launcher
usb 1-1: Manufacturer: Syntek
hid-generic 0003:2123:1010.0001: device has no listeners, quitting
```

 /axi_gpio_0	S_AXI	Reg	0x4120_0000		64K	▼	0x4120_FFFF
 /axi_gpio_1	S_AXI	Reg	0x4121_0000		64K	▼	0x4121_FFFF
 /axi_gpio_2	S_AXI	Reg	0x4122_0000		64K	▼	0x4122_FFFF
 /axi_gpio_3	S_AXI	Reg	0x4123_0000		64K	▼	0x4123_FFFF
 /axi_vdma_0	S_AXI_LITE	Reg	0x4300_0000		64K	▼	0x4300_FFFF
 /v_tc_0	ctrl	Reg	0x43C0_0000		64K	▼	0x43C0_FFFF

Controlling Simple Peripherals

0x41200000 corresponds to the LEDS

Devmem 0x41210000 corresponds to the buttons

devmem 0x41220000 corresponds to the switches

LEDfun.sh can be seen below:

```
SWITCHES=0x41200000
BUTTONS=0x41210000
LEDS=0x41220000

while true; do
    SW=$(devmem $SWITCHES)
    BTN=$(devmem $BUTTONS)

    if [ "$BTN" = "0x00000000" ]; then
        # No buttons pressed - LEDs reflect switches
        devmem $LEDS 32 $SW

    elif [ "$BTN" = "0x00000001" ] || [ "$BTN" = "0x00000002" ] || \
        [ "$BTN" = "0x00000004" ] || [ "$BTN" = "0x00000008" ]; then
        # Exactly 1 button pressed - LEDs reflect inverse of switches
        INV=$(( ~$SW & 0xFF ))
        devmem $LEDS 32 $INV

    else
        # More than 1 button pressed - all LEDs on
        devmem $LEDS 32 0xFF
    fi

    sleep 0.1
done
```

USB Driver Development

In your write-up, summarize the changes that you've made.

The generic `USB_SKEL_VENDOR_ID` and `USB_SKEL_PRODUCT_ID` were replaced with `LAUNCHER_VENDOR_ID` (0x2123) and `LAUNCHER_PRODUCT_ID` (0x1010). All references to `skel` and `usb_skel` were renamed to `launcher` and `usb_launcher`, respectively, including the device struct, driver struct, class registration, and all function names. The launcher has no bulk-out endpoint, only an interrupt-in endpoint. The probe function's bulk-out endpoint discovery and the associated error check were commented out, since attempting to find a bulk-out endpoint would cause the driver to fail to probe the device.

The `launcher_write()` function was significantly modified. The original skeleton used a bulk URB for writes. Since the launcher uses control transfers, the bulk URB allocation and submission were replaced with a `usb_control_msg()` call. The function reads one command byte from userspace and constructs an 8-byte control message with byte 0 set to `LAUNCHER_CTRL_CMD_PREFIX` (0x02), byte 1 set to the command, and the remaining bytes set to 0. The `WRITES_IN_FLIGHT` semaphore was commented out in `launcher_write()`, since the driver only handles a single-character device synchronously, and the concurrency limit added unnecessary complexity. `MODULE_AUTHOR` and `MODULE_DESCRIPTION` were set to Matthew's ISU email and a description identifying this as a USB Missile Launcher driver.

In your writeup, describe how `launcher_fire.c` generally works

```
#define LAUNCHER_NODE        "/dev/launcher0"
#define LAUNCHER_FIRE        0x10
#define LAUNCHER_STOP        0x20
#define LAUNCHER_UP          0x02
#define LAUNCHER_DOWN        0x01
#define LAUNCHER_LEFT        0x04
#define LAUNCHER_RIGHT       0x08
#define LAUNCHER_UP_LEFT     (LAUNCHER_UP | LAUNCHER_LEFT)
#define LAUNCHER_DOWN_LEFT   (LAUNCHER_DOWN | LAUNCHER_LEFT)
#define LAUNCHER_UP_RIGHT    (LAUNCHER_UP | LAUNCHER_RIGHT)
#define LAUNCHER_DOWN_RIGHT  (LAUNCHER_DOWN | LAUNCHER_RIGHT)
```

At the top of our file, we created all these macros that map the bits that move the cannon to their labeled directions. The bits can be OR'd together to perform dual operations.

```
static void launcher_cmd(int fd, int cmd) {
    int retval = 0;
```

```

retval = write(fd, &cmd, 1);
while (retval != 1) {
    if (retval < 0) {
        fprintf(stderr, "Could not send command to %s (error %d)\n", LAUNCHER_NODE, retval);
    }

    else if (retval == 0) {
        fprintf(stdout, "Command busy, waiting...\n");
    }
}
if (cmd == LAUNCHER_FIRE) {
    usleep(5000000);
}
}

```

This launcher_cmd function sends a single byte to the device in order to tell it to complete a task/movement. If the command sent is to launch the cannon, the code sleeps for 5 seconds in order to give enough time for the command to execute.

```

int main() {
    char c;
    int fd;
    int cmd = LAUNCHER_STOP;
    char *dev = LAUNCHER_NODE;
    unsigned int duration = 500;

    cmd = LAUNCHER_FIRE;

    fd = open(dev, O_RDWR);
    if (fd == -1) {
        perror("Couldn't open file: %m");
        exit(1);
    }

    launcher_cmd(fd, cmd);
    usleep(duration * 1000);
    launcher_cmd(fd, LAUNCHER_STOP);
    close(fd);
    return EXIT_SUCCESS;
}

```

The code above is the main function. It initialized the command to stop the launcher, then immediately overrode it with the launcher fire command. The code then connects to the cannon

with read and write access. It sends the launcher command (which includes waiting 5 seconds). Sleeps for another 500ms, and then sends the stop command and closes the device.

Sentry System

In your write-up, describe the general algorithm you implemented

The sentry system combines the camera's real-time video feed with the USB missile launcher to create an automated targeting system. When Switch 5 is enabled, the system enters object detection mode and continuously scans each incoming frame for red objects.

The algorithm operates in the YCbCr color space, which our Vivado outputs. Because red objects produce high Cr (red-difference) and low Cb (blue-difference) chroma values, the system applies a simple two-condition threshold ($Cr > 150$ and $Cb < 120$) to classify each pixel as red or not. Pixels are processed in pairs due to the 4:2:2 chroma subsampling format, where two adjacent pixels share a single Cb/Cr pair.

For every frame, the algorithm accumulates the x and y positions of all qualifying red pixels and divides by the total count to compute a centroid, the average position of the detected object in the frame. This gives a single (x, y) coordinate that represents the red object's location.

That centroid is then compared with the frame center (960, 540), with a 75-pixel buffer dead zone on each axis. If the object is too far left or right, or too far up or down, the system outputs the appropriate directional command to steer the launcher toward the target. Once the centroid falls within the dead zone on both axes simultaneously, meaning the launcher is aimed directly at the object, the system outputs a fire command. If no red pixels are detected in a frame, the system reports that no object was found and continues scanning on the next loop iteration.